

SGPR - Sistema de Gestión de Permisos y Reposos

Es una aplicación Django para gestionar las solicitudes de permisos y reposos del personal, con control de usuarios, auditoría y exportación de reportes.

Contenido

> Nota: no existe un archivo ``readme.py`` en este proyecto. La documentación principal se mantiene en ``README.md``.

- Requisitos (`#requisitos`)
- Instalación local (`#instalaci%C3%B3n-local`)
- Variables de entorno (`#variables-de-entorno`)
- Ejecutar el proyecto (`#ejecutar-el-proyecto`)
- Estructura del proyecto (`#estructura-del-proyecto`)
- Rutas principales (`#rutas-principales`)
- Flujo de uso (`#flujo-de-uso`)
- Notas de desarrollo (`#notas-de-desarrollo`)

Requisitos

- Python 3.10+ (recomendado)
- pip
- SQLite (predeterminado) o PostgreSQL si se configura ``DATABASEURL``
- Node/NPM no son necesarios para correr el backend básico

Instalación local

1. Clona el repositorio o copia el proyecto en tu máquina.
2. En la carpeta raíz del proyecto (``sgpralquitrana``), crea y activa un entorno virtual:
3. Instala las dependencias:
4. Si quieres, copia el ejemplo de variables de entorno:

Variables de entorno

El proyecto carga algunos valores desde variables de entorno. Los valores predeterminados son seguros para desarrollo local.

- ``DJANGOSECRETKEY``: clave secreta de Django.
- ``DJANGODEBUG``: ``True`` o ``False``.
- ``DJANGOALLOWEDHOSTS``: hosts permitidos separados por espacios.
- ``DATABASEURL``: si se define, el proyecto usará esa base de datos en lugar de SQLite.
- ``EMAILHOST``, ``EMAILPORT``, ``EMAILHOSTUSER``, ``EMAILHOSTPASSWORD``, ``EMAILUSETLS``, ``EMAILBACKEND``: para envío de correos.
- ``DEFAULTFROMEMAIL``: emisor del correo.
- ``FERNETKEY``: clave para cifrado y descifrado de campos sensibles en la base de datos (``Auditoria.detalles``, ``Solicitud.motivo``, ``Trabajador.cedulaencrypted``).

Para generar una clave segura, usa:

Si ``EMAILHOST`` no está configurado, el sistema usa el backend de consola (``console.EmailBackend``) y

el correo se imprimirá en la terminal en lugar de enviarse realmente.

El proyecto también carga automáticamente un archivo `.env` en la raíz del proyecto si existe.

Ejecutar el proyecto

1. Ejecuta migraciones:

2. Crea un superusuario si lo deseas:

3. Levanta el servidor:

4. Accede a la aplicación en:

Estructura del proyecto

- `manage.py`: comando principal para ejecutar Django.
- `requirements.txt`: dependencias del proyecto.
- `sgpralquitrana/`: configuración principal de Django.
- `settings.py`: configuración de base, rutas, base de datos y correo.
- `urls.py`: rutas globales del proyecto.
- `gestionpermisos/`: aplicación principal.
- `models.py`: definición de los modelos `Trabajador`, `Solicitud` y `Auditoria`.
- `views.py`: lógica del flujo principal y vistas.
- `forms.py`: formularios de validación y presentación de datos.
- `urls.py`: rutas internas de la aplicación.
- `middleware.py`: middleware para obligar cambio de contraseña cuando sea necesario.
- `templates/`: plantillas HTML que renderiza Django.
- `static/`: archivos estáticos CSS, imágenes y recursos.
- `media/`: archivos subidos por los usuarios (adjuntos).

Rutas principales

Ruta	Qué ofrece
<code>/login/</code>	Login para todos los usuarios
<code>/logout/</code>	Cerrar sesión
<code>/</code>	Panel de solicitudes
<code>/registro/</code>	Registro de trabajador
<code>/solicitar/</code>	Envío de solicitud de permiso/repo
<code>/cambiar-contrasena/</code>	Cambio de contraseña
<code>/trabajadores/</code>	Listado de trabajadores (admin)
<code>/auditoria/</code>	Reporte de auditoría (admin)
<code>/trabajadores/descargar/xlsx/</code>	Exportar trabajadores a Excel
<code>/trabajadores/descargar/pdf/</code>	Exportar trabajadores a PDF
<code>/auditoria/descargar/xlsx/</code>	Exportar auditoría a Excel
<code>/auditoria/descargar/pdf/</code>	Exportar auditoría a PDF
<code>/api/trabajadores/registro/</code>	API JSON para registrar un trabajador

API de registro de trabajadores

La aplicación ahora expone una API simple para crear trabajadores desde un cliente JSON. Esta API registra al trabajador, crea el usuario `username=cédula`, utiliza `DEFAULTPASSWORD` como contraseña inicial y la muestra en pantalla.

- Método: `POST`
- URL: `/api/trabajadores/registro/`
- Payload JSON:
- `cédula`
- `nombres`
- `apellidos`
- `cargo`
- `departamento`
- `email`

> Nota: el API valida que no exista ya un usuario con la misma cédula ni con el mismo correo electrónico.

Ejemplo:

Flujo de uso

1. El usuario puede registrarse o iniciar sesión.
2. Un trabajador envía una solicitud con motivo y adjunto.
3. Gestión Humana revisa la solicitud:
 - Aprueba o rechaza con observaciones.
 - El botón de aprobar/rechazar no aparece si la solicitud ya está procesada.
4. Todas las acciones importantes se guardan en la tabla de auditoría.
5. El personal autorizado puede descargar reportes XLSX/PDF.

Notas de desarrollo

- La validación de archivos limita a 5MB y solo permite PDF e imágenes.
- El campo `adjunto` se guarda en `media/justificantes/`.
- El middleware obliga al primer cambio de contraseña para los trabajadores que reciben contraseña por defecto.
- El sistema usa SQLite local por defecto, pero puede apuntar a PostgreSQL con `DATABASEURL`.

Seguridad y despliegue en producción

Recomendaciones mínimas para asegurar la aplicación antes de exponerla a Internet:

- Coloca en variables de entorno (no en el código) las claves sensibles:
- `DJANGOSECRETKEY` (obligatorio en producción)
- `DATABASEURL` (usar Postgres/managed DB en producción)
- `EMAILHOST` / `EMAILHOSTUSER` / `EMAILHOSTPASSWORD` si usas SMTP
- `FERNETKEY` (clave para cifrado de campos sensibles)

Nota: por seguridad el sistema no muestra contraseñas por defecto en pantalla ni en mensajes. Configure `DEFAULTPASSWORD` solo para entornos controlados o prefiera generar contraseñas temporales y notificar mediante correo con enlace de restablecimiento.

- En `requirements.txt` se añadieron `cryptography` (cifrado de campos) y `argon2-cffi` (hasher opcional). Para usar Argon2, exporta `USEARGON2=True`.
- En `sgpralquitrana/settings.py` hay variables que debes ajustar en producción:
- `DEBUG=False`
- `DJANGOALLOWEDHOSTS` con los dominios/hosts donde se desplegará
- `SESSION\_COOKIE\_SECURE=True`, `CSRF\_COOKIE\_SECURE=True`, `SECURE\_SSL\_REDIRECT=True`
- `SECURE\_HSTS\_SECONDS` (ej. `31536000`) y `SECURE\_HSTS\_INCLUDESUBDOMAINS=True` cuando uses HTTPS
- Genera una `FERNETKEY` segura con:

Luego exporta la clave en la máquina/contenedor que ejecute la app.

- Usa HTTPS para exponer la app (Nginx + Gunicorn o servicios administrados). No expongas directamente `runserver`.

- Recomendación de despliegue: configura variables en un ``.env`` y despliega con Gunicorn + Nginx; para desarrollo use ``python manage.py runserver``.

Comentarios para otros desarrolladores y o mantenedores de la aplicación

- Revisa ``gestionpermisos/views.py`` para entender el comportamiento principal.
- ``gestionpermisos/forms.py`` contiene validaciones clave de datos y archivos.
- ``gestionpermisos/models.py`` calcula días continuos y laborables dentro de ``Solicitud``.
- ``templates/base.html`` administra el layout, el menú y los botones globales.
- Para agregar un nuevo informe, extiende las vistas y agrega una ruta en ``gestionpermisos/urls.py``.